claude-windows / 144a31b7-7962-4fee-9d6f-8a7854daa63f

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\144a31b7-7962-4fee-9d6f-8a7854daa63f\subagents\workflows\wf_134043cd-8c3\agent-a29ce030fc623f452.jsonl`
- SHA-256: `084dd8fa7c292071e908101db2847ae30480201968f65ef54c239049f1bc52ab`
- Source modified: `2026-06-20T09:01:47+00:00`
- Imported at: `2026-07-05T16:48:28+00:00`
- project: `wf_134043cd-8c3`
- session_id: `144a31b7-7962-4fee-9d6f-8a7854daa63f`

Transcript

1. user (2026-06-20T08:59:23.237Z)

```
IMPORTANT grounding (verified this session): the engine is now DECOUPLED ? pluggable
StorageBackend (local + S3/GCS/DO Spaces) in backend/storage/, and a worker `python -m
backend.worker {infer|train}` (backend/worker/__main__.py) that pulls from storage -> runs the
pipeline -> pushes results. Validated e2e on the CPU droplet 168.144.126.176 reading/writing DO
Spaces with the GPU MOCKED (stub detector, RALLY_DETECTOR_IMPL=stub). BOTH the khelsutra site
(/srv/khelsutra-site) AND the engine (~/rally) already live on that ONE droplet. The droplet is
CPU-only (no GPU) and a SHARED testbed (MinIO 9000/9001, GCS emu 9023) ? do not propose touching
those. Real GPU WASB (NativeWasbRunner, "M3.5") is the NEXT build, not done. The FastAPI has NO
auth (CORS '*'). EXCLUDE any path under .claude/worktrees/ ? those are stale copies; read
canonical docs/ + backend/ only.

Map the khelsutra UI/site repo and doc conventions (cwd = C:\\Users\\avidu\\Projects\\khelsutra;
the engine repo is at C:\\Users\\avidu\\Projects\\badminton-highlight-indexer). Read (in
khelsutra) README.md, web/README.md, docs/PER_RALLY_SELECTION.md (just merged),
docs/PORTABILITY.md, site/ structure; and (in engine) docs/PROJECT_DOC_TEMPLATE.md for the
tracked-doc structure. Report: the khelsutra UI surface and how an appliance "operator console"
UI would relate to the public marketing site (gated app vs public page); how this appliance
relates to the per-rally selection doc (per-rally is ONE screen of the console); the exact
tracked-doc template to follow; and concrete reuse opportunities. Quote path:line. Return the
schema.
```

2. assistant (2026-06-20T08:59:25.667Z)

```
I'll map the khelsutra repo and the engine doc conventions. Let me start by pulling fresh and
reading the key files in parallel.
```

3. user (2026-06-20T08:59:31.180Z)

```
Already up to date.
---BRANCH---
master
---LATEST---
eeb8a48 ci(site): add deploy-config + static-asset guards and a live-parity workflow (#12)
cb74ce5 docs(web): tracked design for interactive per-rally selection (RallyPicker) (#11)
0f555ad fix: deterministic LF line endings + live-parity check (droplet vs Cloudflare drift)
(#10)
2c5159d docs: refresh NEXT_STEPS to shipped state + capture new product ideas (#9)
8591ee3 feat(site): email-on-registration via Email Routing send binding (best-effort, portable)
(#8)
```

4. user (2026-06-20T08:59:31.203Z)

```
1       <!--
2       PROJECT DOC TEMPLATE ? copy this file to docs/<YOUR_PLAN>.md and fill it in.
```

```
A "tracked project doc" is how this repo carries substantial design/project work: it can be
ITERATED on, its progress TRACKED, and it is MARKED DONE against explicit deliverables, then
archived. Standardizes the emergent pattern in docs/RALLY_QUALITY_RESEARCH.md §7 and
docs/GOLDEN_REGRESSION_FIXTURES.md.

Rules of thumb:
- §7 (the progress tracker) is the SOURCE OF TRUTH ? one row per independently-shippable PR.
- Keep it HONEST: tag each claim [verified] (checked against code), [researched] (needs sign-off),
  or [design] (proposed). The doc reflects real shipped state, never aspirational.
- It POINTS to detail (code anchors, research artifacts), it does not duplicate it.
- Sections marked OPTIONAL are included only when relevant (e.g. Foundation/Threat-model for
  research- or security/privacy-heavy work). Delete sections that do not apply.
- Move to docs/archives/ once Status = DONE (per the archive policy: only terminal-state work).
-->

# <Project / Plan Title>

> **Status:** `DRAFT` ? `<owner-gated? | in progress>` · **Owner:** `<name>` · **Created:** `<YYYY-MM-DD>` · **Last updated:** `<YYYY-MM-DD>`
> **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when DONE)
> **Tracking anchors:** §7 progress tracker is the source of truth; indexed in `docs/README.md`; pointer in memory `current-state` (and `docs/NEXT_STEPS.md` once work starts).
> **Relation to existing docs:** `<extends | supersedes | peer-of ?>`
> **Honesty note:** mark each claim `[verified]` / `[researched]` / `[design]`. Not legal/financial advice where applicable.

---

## 0. TL;DR
<2?5 sentences: the goal, the approach, and the single most important caveat. Call out any finding that is already actionable on its own.>

## 1. Problem & goal
<Why now; the concrete user-facing outcome; what "good" looks like. Link the code/docs a reader must read to get current.>

## 2. Decisions locked
<Table ? capture answered questions so they are not relitigated.>

| # | Decision | Source / date | Implication |
|---|----------|---------------|-------------|
| D1 | ? | ? | ? |

## 3. Foundation ? research / prior art  *(OPTIONAL)*
<Include when the plan rests on external facts: cited research, legal/compliance, benchmarks, prior-art survey. Mark researched-vs-verified; list any sign-off gates this creates.>

## 4. Design / architecture
<The actual plan. Data contracts, diagrams, and a REUSE MAP ? name the existing modules/code this builds on so it is clear what is new vs reused.>

## 5. Threat model / risk table  *(OPTIONAL ? security / privacy / compliance work)*
<Who/what is protected vs NOT; residual risks stated plainly.>

## 6. Honest limits ? what this does NOT do
<The false-confidence guardrail: what a green suite / a DONE status does NOT prove.>
```

```

## 7. Deliverables & progress tracker   ? **source of truth**
Legend: ? Todo · ? In progress · ? Done · ? Blocked/gated. **One small PR per row**, branched from `origin/master`, no stacking. Update the row (status + PR link) as work lands.

| ID | Deliverable | Depends on | Gated? | Status | PR |
|----|-------------|-----------|--------|--------|----|
| P0 | ? | ? | No | ? | ? |

## 8. Open questions ? owner / external  *(split blocking-gates from nice-to-knows; name who decides)*
1. ?

## 9. Definition of done
<Explicit acceptance checklist. When all ticked ? flip `Status ? DONE` and archive.>
- [ ] ?

## 10. References
**Internal code anchors `[verified]`:** `<paths>`. **Internal docs:** `<docs>`. **External:** `<cited sources>`.

---

### Changelog
- `<YYYY-MM-DD>` ? `<what changed>`.

```

5. user (2026-06-20T08:59:31.212Z)

```
# KhelSutra Guru ?

**Private product repo** ? the heart and brain of [khelsutra.guru](https://khelsutra.guru):
the web app (alpha SPA) now, the hosted control plane at beta.

- **Engine** (pipeline, API, eval): sibling repo `badminton-highlight-indexer` (rename pending).
- **Canonical planning docs** (live in the engine repo's `docs/` for now, migrate here at beta):
  `UI_PROPOSAL.md` · `HOSTING_PLAN.md` · `UI_ALPHA_EXECUTION_PLAN.md`.
- **Status:** pre-alpha scaffold. SPA lands in `web/` per execution-plan milestone M1.

## Structure

- `site/` ? public marketing landing page (static; served at khelsutra.guru)
- `web/` ? React SPA (alpha; Vite + React + TS + Tailwind; Cloudflare Pages build root)
- `docs/` ? product docs as they migrate from the engine repo
- `control-plane/` ? (beta) Cloud Run FastAPI control plane

## License

All rights reserved (private; no OSS license).

```

6. assistant (2026-06-20T08:59:36.156Z)

Now let me read the web/ docs, the per-rally selection doc, portability, and explore the site/ and web/ structure.

7. user (2026-06-20T08:59:36.654Z)

```
SPA scaffold lands here at execution-plan milestone M1 (Vite + React + TS + Tailwind).
Dev: talks to the engine at http://localhost:8000 via Vite proxy; no auth locally (DEV_USER_EMAIL).
```

3

8. user (2026-06-20T08:59:37.602Z)

1 # Interactive per-rally selection ? reel builder (RallyPicker)
2
3 > **Status:** `IN PROGRESS` ? design **APPROVED by owner 2026-06-20**; building the MVP (P1?P5). Downstream milestones (multi-video auth, per-rally preview endpoint, any `/api/compile` default change) remain owner-gated. · **Owner:** Avi · **Created:** 2026-06-20 · **Last updated:** 2026-06-20
4 > **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when DONE)
5 > **Tracking anchors:** §7 progress tracker is the source of truth; indexed in `docs/README.md`; pointer in `NEXT_STEPS.md` + memory `current-state` once work starts.
6 > **Relation:** second major tracked work item in the `khelsutra` repo (peer of `REQUEST_ACCESS_FORM.md`); mirrors the engine repo's `PROJECT_DOC_TEMPLATE` discipline. Primarily realizes alpha screen **A5 Highlights** (engine repo `UI_PROPOSAL.md:47`, exec-plan **M4** `UI_ALPHA_EXECUTION_PLAN.md:39`); its correction-loop extension realizes **A4 Rally Review** (`UI_PROPOSAL.md:46`, exec-plan **M3** `:38`). Consumes the engine's `/api/query` + `/api/compile` contract.
7 > **Honesty note:** every engine-fact claim is tagged `[verified]` (read against engine code this session), `[design]` (proposed here), or `[researched]` (needs sign-off). Not legal/financial advice.
8
9 ---
10
11 ## 0. TL;DR
12 Build the **pick-rallies ? reel** surface as a **hybrid**: a selectable rally **list/grid** is the workhorse; a **deterministic query bar** ("rallies over 10s, longest first") pre-selects into the **same** selection set; a sticky footer compiles the selection into one downloadable MP4. The entire MVP runs on **existing engine endpoints** ? `POST /api/query` ? `POST /api/compile` ? `GET /api/highlights` `[verified backend/main.py:331-340, 342-396, 307-328]` ? with **zero new engine intelligence and zero new endpoints**. The single most important caveat: queries and chips are scoped to **length / count / order / shot-count only**; shot-type, player, and score are **roadmap-only** (no detector, no column) and must not be faked. The biggest UX-vs-effort tension: **no per-rally preview is possible today** ? only the *final compiled reel* can stream ? so MVP selects on text metadata, and visual preview/timeline is a deliberate later milestone.
13
14 ## 1. Problem & goal
15 Today the alpha can detect rallies and stitch a reel, but reel-building is either a **silent `stitching.min_shot_count` threshold** `[verified backend/main.py:362-383]` or a static checkbox list whose data plumbing is even broken ? the bundled static UI reads `data.rallies` while the API returns `play_segments`, so it renders nothing `[verified backend/api/static/app.js:344 vs backend/main.py:340]`. The owner's framing is **dynamic per-video selection, not a fixed threshold**: a coach should say "give me the long rallies from this session" and get exactly those, then fine-tune by hand.
16
17 **The concrete outcome ("good"):** A coach uploads a 40-minute session; the engine finds 60 rallies. They type *"top 15 longest"*, eyeball the auto-selected cards, untick two warm-up rallies, name it *"Tuesday ? long points"*, and download a ~6-minute reel ? without learning any threshold knob.
18
19 **Goal:** the smallest *honest* surface that turns engine-detected rallies into a user-curated, downloadable reel, with both **fast** (query) and **precise** (checkbox) selection sharing **one** state.
20
21 **Non-goals (alpha):** no conversational LLM (`UI_PROPOSAL.md:66` defers it); no shot-type/player/score filters (no data exists); no per-rally video preview while it requires unbuilt streaming; no multi-tenant auth (single-box alpha).
22
23 ## 2. Decisions locked
24 | # | Decision | Source / date | Implication |
25 |---|----------|---------------|-------------|
26 | D1 | Dominant surface = selectable rally **list/grid** | Synthesis 2026-06-20 (Grid

scored 15, fit-to-data 4) | Scales to long matches via bulk+filter; shows honest fields only |
27 | D2 | Fast entry = **deterministic** client-side `parseQuery()` (no LLM) | `[verified UI_PROPOSAL.md:66]` conversational QA deferred | Predictable, unit-testable, offline, zero API cost |
28 | D3 | **One** shared `Set<segment_id>`, mutated by query + checkbox + (future) timeline | Synthesis 2026-06-20 | No second compile path can diverge from what the user sees |
29 | D4 | Compile via existing `POST /api/compile {video_id, segment_ids, output_filename}` | `[verified backend/main.py:342-396]` | Selection order is harmless ? stitcher `_merge_overlapping` de-dupes `[verified compiler.py:80-105, 311-317]` |
30 | D5 | Query grammar (alpha) = length, count, order, **shot_count only** | `[verified database.py:122-135, gemini.py:470-485]` | Only fields the payload actually returns; grey out the rest |
31 | D6 | MVP backend = **zero new endpoints** | `[verified]` reuse map (§4) | Ships the full loop on the already-shipped contract |
32 | D7 | Replace the silent `min_shot_count` cut with **explicit user selection** | Owner framing + `[verified main.py:362-383]` | UI *warns* about the floor; it never hides why a rally vanished |
33 | D8 | **Default selection = ON** (curate-by-removal) | Owner 2026-06-20 | Rallies load all-selected (matches today's "all rallies" reel); user removes warm-ups. Footer/empty UX assumes a non-empty start |
34 | D9 | **Preview-before-select is NOT a ship-blocker** for the MVP | Owner 2026-06-20 | MVP selects on text metadata + previews the *final compiled reel*; the source/clip-stream endpoint (per-rally preview) is the next milestone (P9), not MVP |
35 | D10 | **MVP is job-tethered** (carry `result.video_id`); no `GET /api/videos` | Owner 2026-06-20 | Smallest honest slice; multi-video home + identity scoping is P8 (gated) |
36 | D11 | `min_shot_count`: **warn, do NOT override** in the MVP | Owner 2026-06-20 | The UI surfaces the floor; an explicit-selection override of the compile default is deferred (would trigger the Launch Report convention) |
37
38 ## 3. Foundation ? what the engine gives us today `[verified this session]`
39 The whole design rests on the engine's real, already-shipped contract. Read against the engine repo (`badminton-highlight-indexer`):
40
41 - **The rally store is `play_segments`** ? columns `id, video_id, start_time, end_time, duration, server, receiver, metadata` `[verified database.py:124-135]`. `id` is the integer the UI selects and passes to compile; `start_time/end_time/duration` are REAL seconds (duration computed at insert).
42 - **Listing rallies = `POST /api/query {video_id}` ? `{play_segments:[...]}`** ? the *only* rally-list path today; it JSON-parses `metadata` and joins `events` `[verified main.py:331-340, database.py:427-475]`.
43 - **Building a reel = `POST /api/compile {video_id, segment_ids:[int], output_filename}`** ? loads `videos.filepath`, filters all segments to the requested `segment_ids`, applies the `min_shot_count` floor, extracts `time_pairs=[(start,end)]`, and calls `VideoCompiler.compile_highlights(raw_path, segments, name)` `[verified main.py:342-396, compiler.py:303]`, which **merges overlaps** then per-clip re-encodes ? concat ? one MP4 `[verified compiler.py:311-317]`. Selection order/duplicates are therefore safe.
44 - **Downloading = `GET /api/highlights/{video_id}/{filename}`** ? streams the compiled MP4. Compile returns a **server-local filepath, NOT a URL** `[verified main.py:394]`, so the SPA constructs this download URL itself. The compiled reel is the **only streamable video that exists today** `[verified main.py:307-328]`.
45 - **The floor knob is readable** ? `GET /api/config` exposes `stitching.min_shot_count` `[verified main.py:121-123]`, so the UI can warn before a selected rally is silently dropped.
46 - **Job entry** ? `GET /api/jobs/{job_id}` returns `result.video_id` `[verified main.py:274-293]`, which is how the MVP reaches a video without a (nonexistent) list-videos endpoint.
47
48 **Honest fields vs fabricated/roadmap (critical for the UI):**
49 - **Honest today** (default `default_segmenter: "gemini"`, per `config.json` `[verified]`): `start_time`, `end_time`, `duration`, and `metadata.shot_count` (Gemini's `shuttle_exchange_count`, fallback `max(3, int(dur/0.8))`) + `metadata.shot_count_confidence` (a **string**, often `"Unknown"`) `[verified gemini.py:470-485]`. On the gemini path `server`/`receiver` are written `None` ("no fabricated data") `[verified gemini.py:480-487]`.
50 - **Fabricated ? do NOT surface:** the `motion` demo segmenter fills `server`/`receiver` via `random.sample(...)`, a random `shot_count`, and random `events` (serve/smash/foul/winner) `[verified motion.py:191-198]`. `query` returns these columns + `events`, but they are not

ground truth and must stay hidden.
51 - **Roadmap ? no column, no detector:** shot-type per shot, player identity, match/point score, game/set grouping. None exist anywhere in the schema; do not consume them `[verified database.py whole-schema 122-148]`.
52 - **Correction-loop columns exist but are dead:** `candidate_segments.human_label/notes/confirmed_segment_id` are defined but no endpoint writes them `[verified database.py:166-168]` ? that's the M4 corpus extension (engine PR-4/B6, `UI_ALPHA_EXECUTION_PLAN.md`).
53
54 ## 4. Design / architecture
55
56 **Reuse map (existing ? `[verified]`, no change for MVP):** `POST /api/query` (list) ? `POST /api/compile` (stitch) ? `GET /api/highlights/{video_id}/{filename}` (download); `GET /api/config` (floor) ; `GET /api/jobs/{id}` (carry `result.video_id`). **All net-new code is the SPA** in `khelsutra` `web/` (Vite + React + TS + Tailwind, per `web/README.md`), plus a small set of *optional* real-gap engine enablers (§4.3) ? none required for MVP.
57
58 **4.1 Components (net-new SPA)**
59 - **QueryBar + `parseQuery()`** ? a *pure, deterministic, unit-tested* string ? `{filter, sort}` compiler over `duration` / `ordinal` / `shot_count`. Renders a **parsed-pill** ("filter: duration > 10s · sort: duration desc") so the interpretation is transparent and hand-correctable; greys out shot-type/player/score tokens with an inline "not available yet". No LLM, no network call.
60 - **RallyList/Grid** ? one card per `play_segment`: ordinal (by `start_time`), `mm:ss` start, duration (`end?start`), and a shot-count + confidence chip **only when `metadata.shot_count` is present** (absent on detector paths that don't write it). Checkbox per card; the thumbnail area is a placeholder (no endpoint yet).
61 - **`useRallySelection()` hook** ? owns `selectedIds:Set<number>`, `displayOrder`, `apply(queryResult)`, `toggle(id)`, and bulk Select-all / Clear / Invert. Unit-tested (matches the exec-plan's "API client + review-state logic" bar).
62 - **SelectionFooter (sticky)** ? live "N clips / total M:SS"; a **`min_shot_count` warning** when a selected rally would be dropped at compile (read from `GET /api/config`); reel-name input; one **Build** button.
63 - **CompileResult** ? `<video src=GET /api/highlights/...>` preview of the *compiled reel* + a download link.
64 - **TypedApiClient** ? `query`, `compile`, `buildDownloadUrl`; the typed seam to the engine.
65
66 **4.2 Data flow** `[verified anchors inline]`
67 Gemini segmenter (process time) writes `play_segments {id, start/end_time, duration, metadata.shot_count/confidence/detector}` `[gemini.py:461-498]`
68 ? SPA `POST /api/query {video_id}` ? `{play_segments:[...]}` `[main.py:331-340]` (**fix** the static UI's `data.rallies` read ? `play_segments` `[app.js:344]`)
69 ? normalize to `RallyCard[]` + one shared `selectedIds:Set`; `server`/`receiver`/`events` are present in the payload but motion-only fabrications ? **not surfaced** `[motion.py:191-198]`
70 ? all input modes mutate the **same** `selectedIds`: (a) `parseQuery()` predicate over duration/ordinal/shot_count; (b) checkbox toggle + bulk ops; (c) *(post-MVP)* timeline band
71 ? footer recomputes count + ?duration **client-side**, and warns for any selected rally with `shot_count < stitching.min_shot_count` `[main.py:362-383]`
72 ? **Build** ? `POST /api/compile {video_id, segment_ids: [...selectedIds], output_filename}` ? stitcher de-dupes via `_merge_overlapping` `[compiler.py:311-317]`, cuts `time_pairs` from `videos.filepath`, re-encodes per clip ? concat ? one MP4 `[main.py:386-393, compiler.py:303]`
73 ? compile returns a **server-local filepath, not a URL** `[main.py:394]` ? SPA builds `GET /api/highlights/{video_id}/{output_filename}` to preview in `<video>` + download `[main.py:307-328]`.
74
75 **4.3 Engine API additions (none for MVP; tagged against real gaps)**
76 | Endpoint | Purpose | Status |
77 |---|---|---|
78 | `POST /api/query` | List rallies (only path) ? fix SPA to read `play_segments` | **exists** |
79 | `POST /api/compile` | Stitch selected `segment_ids` ? MP4 | **exists** |
80 | `GET /api/highlights/{video_id}/{filename}` | Stream/download compiled reel |

exists |
`GET /api/config`	Read `stitching.min_shot_count` for the warning	**exists**
`GET /api/jobs/{job_id}`	Carry `result.video_id` into the picker	**exists**
`POST /api/compile` ? add `download_url`	Return `/api/highlights/...` so SPA stops hand-building it (additive, low-risk; today returns only a filepath `main.py:394`)	**modify**
`GET /api/health`	Engine-offline banner ping the SPA is specified to use (`HOSTING_PLAN.md:42`)	**new**
`GET /api/videos/{video_id}/rallies`	RESTful/cacheable alias returning the same `{play_segments}` shape	**new**
`GET /api/videos`	List a user's videos for a Match picker (only `get_video(id)` exists, `database.py:418`) ? needs DB enumerate + identity scoping	**new**
`GET /api/videos/{video_id}/source` (Range) *or* `/clip?start=&end=`	Stream source/per-span video so a rally previews before selection ? neither exists	**new**
`GET /api/videos/{video_id}/rallies/{segment_id}/thumbnail`	Per-rally poster still (no thumbnail column; WASB frames are transient)	**new**
`PATCH /api/segments/{id}`	Persist accept/reject/nudge ? `candidate_segments.human_label/notes/...` (columns exist, unwritten ? corpus loop)	**new**

? Dead-ends / do-NOT: never surface `server`/`receiver`/`events` (motion-only randoms `[motion.py:191-198]`); never consume shot-type/player/score (no data); never render a confidence **meter** ? `shot_count_confidence` is a Gemini *string* (often `"Unknown"`) and `play_segments` has **no** numeric confidence column.

5. Risk table

Risk	Severity	Mitigation
No source/clip/frame/thumbnail endpoint ? only compiled reels stream `[main.py:307-328]`	High (UX)	MVP selects on text metadata + previews the *final* reel; preview/timeline deferred to M3 behind a new endpoint
`min_shot_count` silently drops selected rallies `[main.py:362-383]`	High (looks broken)	Footer warns *before* Build; D7 makes selection explicit; an override is an owner gate (Launch-Report-worthy `[memory: launch-report-convention]`)
No match-duration field ? a timeline must derive `total = max(end_time)`, mis-rendering pre/post dead air	Med	Timeline deferred to M3; never over-promise it as a true scrubber until source streams
`shot_count` is an *estimate* (`shuttle_exchange_count` w/ `max(3,int(dur/0.8))` fallback) `[gemini.py:470-485]`	Med	Label "approx."; confidence shown as the raw string, not a meter
No auth / user scoping anywhere (CORS `*`, `videos.user_id` never set/filtered)	High for multi-tester	Fine for single-box local alpha; a ship-blocker for any shared host ? gate `GET /api/videos` behind PR-3 scoping
compile returns a filepath not a URL `[main.py:394]`	Low	Client builds the URL for MVP; `download_url` field (M1) removes the brittleness
Surface sprawl (query + grid + future timeline + future correction)	Med	Deliberate IA: query+grid = one screen; correction = separate ? corpus screen (owner call, §8)

6. Honest limits ? what this does NOT do

A green SPA build + passing `parseQuery`/selection unit tests prove the **picker logic only** ? **not** that the engine's rally boundaries are correct (the measured **rally-END** weakness is a separate quality track, unaffected by this UI). "Conversational" here means a **deterministic four-dimension parser**, not natural-language understanding. Nothing is implemented in `web/` yet ? this is `[design]`. The feature curates *what the engine already found*; it cannot surface a rally the detector missed, nor split/merge a mis-bounded one (that's the M4 correction loop).

7. Deliverables & progress tracker ? **source of truth**

Legend: ? Todo · ? In progress · ? Done · ? Blocked/gated. **One small PR per row**, branched from `origin/master`, no stacking. CI (`npm test`) must stay green; merge to `master` auto-deploys the site (docs/`web/` changes don't touch the live `site/` Worker).

ID	Deliverable	Depends on	Gated?	Status	PR
P0	This design doc + `docs/README.md` + `NEXT_STEPS.md` pointers	?	No	?	[#11](https://github.com/avidullu/khelsutra/pull/11)

P1	SPA scaffold (Vite+React+TS+Tailwind) in `web/` + typed API client; dev proxy to engine `:8000`	P0	No	?	?	
P2	RallyList/Grid (honest fields only) + `useRallySelection()` hook (unit-tested)	P1	No	?	?	
P3	`parseQuery()` + parsed-pill + example chips (unit-tested; greys out unavailable tokens)	P2	No	?	?	
P4	SelectionFooter + `min_shot_count` warning (from `GET /api/config`)	P2	No	?	?	
P5	Build ? `POST /api/compile` ? client-built download URL + `<video>` preview of the reel	P2	No	?	?	
P6	*(engine repo)* `GET /api/health` + offline banner; `/api/compile` returns `download_url`	?	No	?	?	
P7	*(engine repo)* `GET /api/videos/{id}/rallies` RESTful alias	?	No	?	?	
P8	*(engine repo)* `GET /api/videos` + PR-3 identity scoping ? A1 Matches screen	P7	? owner	?	?	
P9	*(engine repo)* source/clip stream + timeline + per-rally preview/thumbnail	P8	? owner	?	?	
P10	*(engine repo)* `PATCH /api/segments/{id}` corpus loop (PR-4/B6) ? A4 Rally Review	P9	? owner	?	?	

Milestone rollup: **M0/MVP = P1?P5** (pick?reel on existing API, zero new engine endpoints) · **M1 = P6?P7** (RESTful + reliability polish) · **M2 = P8** (multi-video home) · **M3 = P9** (per-rally preview/timeline) · **M4 = P10** (corpus correction loop) · **M5** = swap `parseQuery()` for localized NL behind the same seam (gated on shot/player data landing).

8. Open questions ? owner / external
Blocking gates ? ? RESOLVED 2026-06-20 (owner) ? see D8?D11 in §2:
1. ~~Default selection ON vs OFF?~~ ? **ON / curate-by-removal** (D8).
2. ~~Is preview-before-select a ship-blocker?~~ ? **No**; text-metadata MVP, preview is P9 (D9).
3. ~~Job-tethered vs multi-video MVP?~~ ? **Job-tethered** (D10).
4. ~~`min_shot_count` override?~~ ? **Warn, don't override** in MVP (D11).

Still open (nice-to-knows, non-blocking):
5. May user-facing copy call `parseQuery()` "conversational", given conversational QA is explicitly deferred (`UI_PROPOSAL.md:66`)? Risk of over-promising NL. *Owner positioning/honesty call.* *(Lean: avoid the word; show the parsed-pill instead.)*
6. One screen for query+grid (recommended) vs a separate ? corpus/correction screen for A4? *Owner positioning call.*

9. Definition of done
- [ ] From a finished job, a user can list rallies, query/select, and download a reel ? on **existing** endpoints.
- [ ] Only honest fields (ordinal / duration / shot_count+confidence-when-present) are shown; fabricated `server`/`receiver`/`events` are hidden.
- [ ] `parseQuery()` covers length/count/order/shot_count, greys out unavailable tokens, shows a visible parsed-pill, and is unit-tested.
- [ ] Selection hook is unit-tested; the `min_shot_count` floor is surfaced, not hidden.
- [ ] `npm run build` (and `npm test`) clean; the static UI's `data.rallies` bug is gone (or the static UI is superseded by the SPA).
- [ ] §7 tracker + this DoD updated; doc archived when P1?P5 land (MVP).

10. References
Engine code anchors `[verified this session]`: `backend/main.py:121-123` (config), `:274-293` (jobs), `:307-328` (highlights download), `:331-340` (query), `:342-396` (compile + `min_shot_count` floor + filepath-not-URL); `backend/infrastructure/database.py:124-135` (play_segments), `:166-168` (dead correction columns), `:418` (no list-videos), `:427-475` (query read path + events join); `backend/pipeline/segmenters/gemini.py:461-498, 470-485` (honest shot_count, server/receiver=None); `backend/pipeline/segmenters/motion.py:191-198` (fabricated fields); `backend/pipeline/stitcher/compiler.py:80-105, 303, 311-317` (merge/de-dupe + entrypoint); `backend/api/static/app.js:344` (`data.rallies` bug); `config.json` (`default_segmenter: "gemini"`).
Engine docs: `UI_PROPOSAL.md:46-47, 66`; `UI_ALPHA_EXECUTION_PLAN.md:38-39`; `HOSTING_PLAN.md:42`; `OWNED_MODEL_IMPLEMENTATION_PLAN.md:54-55` (conversational seam built now, model punted). **This repo:** `web/README.md`, `REQUEST_ACCESS_FORM.md`.

````

---
### Changelog
- 2026-06-20 ? Initial `DRAFT`. Consolidated three candidate designs (Timeline / Grid / RallyPicker) into the hybrid recommendation via a multi-agent design pass; every engine-fact anchor re-verified against source before writing.
- 2026-06-20 ? Owner **approved** the recommendations; locked D8?D11 (default ON · preview not a ship-blocker · job-tethered MVP · warn-don't-override the floor); status `DRAFT ? IN PROGRESS`; P0 done.

````

9. user (2026-06-20T08:59:37.691Z)

````
# Portability ? Cloudflare now, always-on VM if needed

> **Status:** design + proven (2026-06-20). **Decision:** keep the site/form host-portable so
> moving off Cloudflare to an owner-run always-on VM ("if Cloudflare acts out") is an **adapter
> swap, not a rewrite.**

## The shape that makes it portable
- **Logic is runtime-agnostic.** `src/index.js` uses Web-standard `Request`/`Response` and pure
  helpers (`validate`, `toRow`, `isBot`, `handleRequest`) ? no host-specific calls.
- **All DB coupling is in adapters** (`src/store.js`): `d1Store` (Cloudflare D1) and `sqliteStore`
  (node:sqlite / better-sqlite3). `handleRequest(request, store)` takes the store by injection.
- **One schema, one SQL.** `schema.sql` and the `INSERT` are plain SQLite, so D1 and a VM's SQLite
  file are byte-for-byte compatible ? zero dialect change. (Postgres later = minor placeholder tweaks.)

## Two hosts, same code
| | Cloudflare (now) | Always-on VM (fallback) |
|---|---|---|
| Entry | `src/index.js` (Worker `fetch`) | `server.js` (`node:http`) |
| Static | `env.ASSETS` | `server.js` serves `public/` |
| Storage | `d1Store(env.DB)` ? D1 | `sqliteStore(db)` ? `node:sqlite` file |
| Country | `cf-ipcountry` header | optional (GeoIP / null) |
| Deploy | git ? Workers Builds | `node server.js` under systemd |
| TLS/DNS | Cloudflare | keep Cloudflare in front, **or** Caddy/Let's Encrypt |

**Enforced, not asserted.** `node --test src/` runs BOTH paths and CI (`.github/workflows/test.yml`,
Node 24) runs them on every PR, so the repo stays compatible with both hosts:
- `src/index.test.js` ? the Cloudflare Worker + D1 logic.
- `src/server.test.js` ? **boots the real `server.js`** (node:http + node:sqlite) and round-trips it
  over HTTP into SQLite (the same proof we run on the droplet, automated).

For a live check against a real VM: `SSH_TARGET=root@host scripts/droplet-smoke.sh` (scp + boot +
round-trip + cleanup). Verified live on the owner's droplet (Ubuntu 24.04, isolated `/srv/khelsutra-site`).

## VM deploy runbook ("if Cloudflare acts out")
Node 22+ only ? **no `npm install`** (node:sqlite is built in).
```
# on the VM:
git clone ? && cd khelsutra/site        # or scp the site/ folder
PORT=8080 DB_PATH=/var/lib/khelsutra/access.db node server.js
# put it under systemd (auto-restart) + a reverse proxy (Caddy/nginx) for TLS,
# then point DNS at the VM. ~1 hour; logic + schema unchanged.
````
````

```
42      ```
43
44      ## Honest notes
45      - "Cloudflare acts out" usually means **Workers/D1 pricing or limits, not DNS** ? so the
cheapest
46        move is to keep Cloudflare DNS/CDN in front and shift only the **compute** to the VM
(point a
47        route/origin at the box). Full exit (own DNS + Caddy TLS) is also supported.
48      - You can run **both** (Cloudflare primary + VM warm standby) since the schema is
identical; a
49        periodic `access_requests` export keeps them in sync.
50      - `server.js` is intentionally dependency-free; if you later want Postgres or a
framework (Hono runs
51        on Workers *and* Node), it slots in behind the same `store` + handler.
52
```

10. assistant (2026-06-20T08:59:40.847Z)

Now let me explore the directory structures of site/ and web/, and the engine's relevant docs (UI_PROPOSAL, HOSTING_PLAN, the storage backend, worker entry point).

11. user (2026-06-20T08:59:43.318Z)

```
=== TOP LEVEL ===
total 30
drwxr-xr-x 1 avidu 197609    0 Jun 20 14:19 .
drwxr-xr-x 1 avidu 197609    0 Jun 18 11:14 ..
drwxr-xr-x 1 avidu 197609    0 Jun 20 14:29 .git
-rw-r--r-- 1 avidu 197609  271 Jun 20 13:05 .gitattributes
drwxr-xr-x 1 avidu 197609    0 Jun 20 12:43 .github
-rw-r--r-- 1 avidu 197609   60 Jun 10 23:12 .gitignore
-rw-r--r-- 1 avidu 197609 3467 Jun 20 14:19 NEXT_STEPS.md
-rw-r--r-- 1 avidu 197609  925 Jun 20 08:37 README.md
drwxr-xr-x 1 avidu 197609    0 Jun 20 14:19 docs
drwxr-xr-x 1 avidu 197609    0 Jun 20 14:22 site
drwxr-xr-x 1 avidu 197609    0 Jun 10 23:12 web

=== site/ ===
site/make_og.py
site/package.json
site/public/capture.html
site/public/index.html
site/public/og.png
site/README.md
site/schema.sql
site/scripts/droplet-smoke.sh
site/scripts/parity-check.sh
site/server.js
site/src/assets.test.js
site/src/deploy-config.test.js
site/src/index.js
site/src/index.test.js
site/src/notify.js
site/src/server.test.js
site/src/store.js
site/wrangler.toml

=== web/ ===
web/README.md

=== docs/ ===
PER_RALLY_SELECTION.md
PORTABILITY.md
README.md
REQUEST_ACCESS_FORM.md
```

```
=== .github/workflows ===
parity.yml
test.yml
```

12. assistant (2026-06-20T08:59:46.053Z)

Let me read the khelsutra docs/README.md, NEXT_STEPS.md, REQUEST_ACCESS_FORM.md, and the engine's UI/hosting docs plus the storage/worker code anchors.

13. user (2026-06-20T08:59:46.545Z)

```
1       # khelsutra docs index
2
3       ## Tracked work (this repo)
4       - **`PER_RALLY_SELECTION.md`** ? interactive pick-rallies ? reel builder (RallyPicker).
The alpha
5         SPA over the engine's rally index; hybrid grid + deterministic query bar on the
**existing**
6         `/api/query` + `/api/compile` contract. `DRAFT` (owner-gated on scope).
7       - **`REQUEST_ACCESS_FORM.md`** ? segmented lead-capture form + Worker/D1 backend
(shipped).
8       - **`PORTABILITY.md`** ? the Node+SQLite VM-host runbook (Cloudflare ? droplet parity).
9
10      ## Engine planning docs
11      Canonical planning docs currently live in the engine repo (`badminton-highlight-
indexer/docs`):
12      `UI_PROPOSAL.md` · `HOSTING_PLAN.md` · `UI_ALPHA_EXECUTION_PLAN.md`. They migrate here
when the
13      control plane lands (per ADR-009 in the engine repo's
`docs/archives/decisions/DECISIONS.md`).
14
```

14. user (2026-06-20T08:59:47.117Z)

```
1       # Next steps (live) ? KhelSutra brand / site
2
3       Prioritized "what to do next" for the **`khelsutra` product repo**. Refresh at each
checkpoint.
4
5       **Where we are (2026-06-20):** `khelsutra.guru` is **LIVE and fully shipped** ? landing
page + `/capture`
6       checklist + OG image + a **segmented Request-Access form** (academy/individual + GPU +
storage + free
7       text). On **Cloudflare** (Worker `hidden-king-khelsutra` + D1 `khelsutra-access`): the
form persists to
8       D1 **and emails a summary to the owner on every registration** (Email Routing send
binding ? verified
9       live). **Git auto-deploy is connected** (Workers Builds, root `site/`): merge to
`master` ? deploys;
10      **CI runs `npm test` (20 green) on every PR**. The same form also runs on a dependency-
free **Node +
11      SQLite** host (`site/server.js`), **live + browsable on the DO droplet ?
http://168.144.126.176:8787**
12      (systemd `khelsutra-site`). PRs #1?#8 merged. Docs: `docs/REQUEST_ACCESS_FORM.md`,
`docs/PORTABILITY.md`.
13
14      ## Next product threads (owner ideas, 2026-06-20)
15      - **Marketing video on the site** ? use Grok/Gemini image-generation to make a short
video (themed for
16        **serious enthusiasts**) walking the whole record ? index ? reel process, and **embed
it** on the
17        landing page. Design-first (storyboard + tone), then produce + embed.
18      - **Interactive per-rally selection** (big product feature) ? the engine detects
rallies; present them
```

  in a UI so the user **picks which rallies go into the highlight** ? dynamic per-video, **not a fixed
  threshold/setting**. Pairs with the "ask for a reel" conversational builder already teased on the
  site. Lives in the alpha SPA (`web/`) over the rally index the engine produces (persisted candidate
  segments). **? Tracked design doc: `docs/PER_RALLY_SELECTION.md` (DRAFT, owner-gated on scope).**
  Key finding: the **MVP needs zero new engine endpoints** ? it rides the existing `POST /api/query`
  ? select `segment_ids` ? `POST /api/compile` ? `GET /api/highlights`. Owner decisions pending in
  the doc's §8 (default selection ON/OFF; is preview-before-select a ship-blocker; job-tethered vs
  multi-video MVP; whether the manual selection overrides the silent `min_shot_count` floor).

Queued (site polish)
- **Always-Use-HTTPS + HSTS** (Cloudflare SSL/TLS) ? closes the plain-HTTP flag.
- **Cloudflare Web Analytics** ? paste the token into `site/public/index.html` (or enable auto-setup).
- **`/capture` footer link** ? currently `/capture.html` (307?`/capture`); change to `/capture`.
- **Droplet ? production** ? TLS + a subdomain (`vm.khelsutra.guru` via Caddy) if it should be a real
  standby; add **VM-path SMTP** so the Node host also emails (currently `noopNotify`).
- **DMARC** TXT for deliverability; delete any orphan `khelsutra-site` Worker; clean CF D1 test rows
  (`deploy-test@`, `email-test@`). Revoke the droplet SSH key (`~/.ssh/khelsutra_droplet`) when done.

Bigger arcs (design-first)
- **Alpha SPA** in `web/` (Vite + React + TS + Tailwind) ? hosts the per-rally selection UI above.
- **BYO-storage + autosync** ? connect Drive/OneDrive/GCS/S3, watch the folder, auto-prepare reels
  (engine `PLATFORM_ARCHITECTURE.md §3`). The form's GPU/storage answers are the demand signal.
- **Control plane** (`control-plane/`) at beta.

Convention
Substantial work gets a tracked doc in `docs/`; one PR per item off `origin/master`; merge to `master`
auto-deploys (CI must be green).

15. user (2026-06-20T08:59:47.385Z)

Request-Access form ? segmented lead capture + setup discovery

> **Status:** `IN PROGRESS` · **Owner:** Avi · **Created:** 2026-06-20 · **Last updated:** 2026-06-20
> **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived`
> **Relation:** first tracked work item in the `khelsutra` repo; mirrors the engine repo's
> `PROJECT_DOC_TEMPLATE` discipline (status header · decisions-locked · §6 tracker · DoD).
> Feeds the platform direction in the engine repo's `PLATFORM_ARCHITECTURE.md §3` (StorageBackend /
> ComputeBackend) and `VIDEO_LOCALITY_MODEL.md` ? see §7.

---

0. TL;DR
Replace the landing page's `mailto:` "Request access" CTA with a real form that (a)

segments **academies vs individual enthusiasts**, (b) asks the two questions that actually decide how we'd serve a user ? **"do you have a GPU machine?"** (compute placement) and **"where do your videos live?"** (storage backend) ? and (c) keeps a free-text message. Submissions are stored in our own **Cloudflare D1** via a small **Worker** (`POST /api/request-access`). The form doubles as PMF + architecture research.

1. Problem & goal
Today the CTA is `mailto:hello@khelsutra.guru` ? zero structure, nothing collected, no segmentation. We want **structured, queryable** interest signals from two distinct audiences, captured the moment someone raises a hand, **without shipping lead data to a third party** (privacy-first ethos).
Goal: a clean on-brand form whose responses land in our own Cloudflare account, queryable later.

2. Decisions locked
- **Backend = Cloudflare Worker + D1** (owner decision, 2026-06-20). Data stays in our Cloudflare account; queryable; no third party. (Alternatives considered: hosted form (Tally/Formspree ? third-party data), Worker?email-only (no DB), structured `mailto` (no collection). Rejected for the reasons in parens.)
- **Two audience segments:** `academy` and `individual` ? the form lightly adapts (academy also asks club name + #courts).
- **The two discovery questions are first-class** (not optional polish): GPU/compute + storage ? they map 1:1 onto the BYO-compute / BYO-storage architecture forks (§7).
- **Free-text message retained** ("Anything else?").
- **Privacy:** store only what's submitted + a coarse country (from `request.cf.country`); **no raw IP**; consent checkbox for "email me about the alpha"; this data is **operations/contact only**, never a training source (same guardrail as the engine repo).
- **Deploy model changes:** the site moves from dashboard direct-upload ? **Wrangler** (Worker with static assets + D1 binding). Documented in §5; owner runs the `wrangler` deploy (our account/auth).

3. Form spec
Field	Type	Notes
Segment	radio	`academy` | `individual` (required)
Name	text	required
Email	email	required (so we can reply)
City	text	optional
Club / academy name	text	shown for `academy`
Courts recorded	select	`1` | `2-4` | `5+` (academy; optional)
GPU machine for local processing?	radio	`yes` | `unsure` | `no` | `prefer-cloud`
Where do your recordings live / how would you share them?	checkboxes (multi)	`gdrive` | `onedrive` | `dropbox` | `bucket` (S3/GCS) | `local` (SD/drive) | `unsure`
Message	textarea	free text ("Anything else?")
Consent	checkbox	"Okay to email me about the KhelSutra alpha"
`hp`	hidden honeypot	anti-spam; if filled ? silently drop

4. Design / architecture
- **Frontend** (`site/index.html`): the header + hero "Request access" buttons scroll to

````
the
  `#join` form; `#join` becomes the form. On submit, JS `POST`s JSON to `/api/request-access`.
  **Progressive enhancement:** if the endpoint isn't live yet (pre-Worker), it falls back to a
  prefilled `mailto:hello@` so the form is useful from the first re-upload.
- **Worker** (`/api/request-access`): validates (email present, honeypot empty), inserts into D1,
  returns `200 {ok:true}`; serves the static assets for all other routes.
- **D1 schema** (`access_requests`): `id, created_at, segment, name, email, city, org_name, courts,
  has_gpu, storage (csv), message, consent, country, user_agent`.
- **Anti-spam:** honeypot first; add **Cloudflare Turnstile** in P2 if spam appears.
- **Notify (P2):** optional email ping to `hello@` per submission (MailChannels / Cloudflare Email).

## 5. Deploy (owner-run; our Cloudflare account)
```
wrangler d1 create khelsutra-access            # once; copy the database_id into wrangler.toml
wrangler d1 execute khelsutra-access --file=site/schema.sql   # create the table
wrangler deploy                                # ships the Worker + static assets + D1 binding
```
> Until the Worker ships, the form works via the `mailto` fallback (re-upload `site/` as today).
> Git?Cloudflare auto-deploy still intentionally not connected.

## 6. Deliverables & progress tracker  ? source of truth
Legend: ? Todo · ? In progress · ? Done. One PR per row off `origin/master`.
| ID | Deliverable | Gated? | Status | PR |
|----|-------------|--------|--------|----|
| P0 | This design doc + `NEXT_STEPS.md` | No | ? | #2 |
| P1 | Form UI (segments + GPU + storage + free text + consent) w/ `mailto` fallback | No | ? | #2 |
| P2 | Worker + D1 + `wrangler.toml` + `schema.sql` + **unit tests (10 green)** | owner deploy | ? | this PR (built+tested; owner runs `wrangler d1 create` + `deploy`) |
| P3 | Anti-spam (Turnstile) + per-submission email notify | No | ? | ? |
| P4 | Tiny admin/export view (auth'd) to read responses | owner | ? | ? |

## 7. Why this matters beyond lead capture
The two questions are **the architecture forks** from the engine repo's BYO analysis:
- **GPU?** ? compute placement: desktop/local (their GPU) vs hosted vs zero-infra-BYO
  (`PLATFORM_ARCHITECTURE.md §3.2`, `DESKTOP_APP_PLAN.md` P0 spike).
- **Storage?** ? which `StorageBackend` + whether autosync is worth building
  (`PLATFORM_ARCHITECTURE.md §3.1`, the cloud-storage/autosync thread).
So early responses directly inform what to build next ? not just who to email.

## 8. Open questions
1. Turnstile now or wait for spam? (Default: honeypot now, Turnstile if needed.)
2. Email-notify per submission, or just query D1 weekly? (Default: notify in P2.)
3. Do we localize the form (Kannada/Hindi) for academies now or at beta? (Default: English now.)

## 9. Definition of done
- [ ] Form replaces the `mailto` CTA, segments academy/individual, asks GPU + storage + free text.
- [ ] Submissions persist to Cloudflare D1 via the Worker; honeypot drops bots.
- [ ] Privacy posture honored (no raw IP; consent; contact-only).
- [ ] Deployed live on khelsutra.guru; a test submission appears in D1.
- [ ] Doc tracker + DoD updated; archived when P1?P3 land.

````

16. assistant (2026-06-20T08:59:51.986Z)

Now let me look at the engine-side docs referenced (UI_PROPOSAL, HOSTING_PLAN) and the storage/worker code anchors to ground the appliance/operator console relation.

17. user (2026-06-20T08:59:56.556Z)

Already up to date.
=== docs UI/HOSTING ===
DECOUPLED_COMPUTE
DESKTOP_APP_PLAN.md
HOSTING_PLAN.md
NEXT_STEPS.md
PLATFORM_ARCHITECTURE.md
UI_ALPHA_EXECUTION_PLAN.md
UI_PROPOSAL.md
VIDEO_LOCALITY_MODEL.md
=== storage/ ===
backend/storage/base.py
backend/storage/gcs.py
backend/storage/gdrive.py
backend/storage/local.py
backend/storage/ref.py
backend/storage/s3.py
backend/storage/__init__.py
backend/storage/__pycache__/base.cpython-313.pyc
backend/storage/__pycache__/gcs.cpython-313.pyc
backend/storage/__pycache__/gdrive.cpython-313.pyc
backend/storage/__pycache__/local.cpython-313.pyc
backend/storage/__pycache__/ref.cpython-313.pyc
backend/storage/__pycache__/s3.cpython-313.pyc
backend/storage/__pycache__/__init__.cpython-313.pyc
=== worker/ ===
backend/worker/__init__.py
backend/worker/__main__.py
backend/worker/__pycache__/__init__.cpython-313.pyc
backend/worker/__pycache__/__main__.cpython-313.pyc

18. assistant (2026-06-20T09:00:00.362Z)

The engine repo cwd is the badminton-highlight-indexer (the Bash tool resets cwd). Let me read the key engine docs and code for the operator-console grounding: UI_PROPOSAL, HOSTING_PLAN, the worker entry, storage base, and the DECOUPLED docs dir.

19. user (2026-06-20T09:00:01.313Z)

```
# UI Proposal ? khelsutra.guru web app (alpha ? beta)

**Status:** PROPOSAL ? awaiting owner ratification (no implementation starts without explicit approval, per CLAUDE.md).
**Date:** 2026-06-10 · **Inputs:** PMF_MARKET_RESEARCH, COMMERCIALIZATION, PLATFORM_ARCHITECTURE, DEFERRED_HARDENING_PLAN (#16), ROADMAP, VIDEO_RECORDING_GUIDELINES, reporting/spec.yaml, current API + static UI.
**Companion:** `docs/HOSTING_PLAN.md` (domain, topology, costs).

## 1. Strategic frame ? why a UI now, and what it secretly is

The UI is not just product surface; it is the **corpus-growth engine**. The single highest-leverage
item in NEXT_STEPS is growing the golden corpus, and the v1 scope already commits to a
**human correction loop**. Every tester who accepts/rejects/nudges a rally boundary in the review
screen produces exactly the labels the Gen-0/M0.x track is starved for (subject to the per-user
training opt-in guardrail). 10?25 alpha testers ? a steady stream of labeled, real-world,
```

multi-court-biased footage ? the distribution where the owned model's 0.66 ship target lives.

Serving economics per the 2026-06-10 quality table: alpha serves the **Gemini path** (0.93 on clean footage, ~$0.05/run) ? no GPU required in the request path. The WASB/owned-model path stays on the owner's box and is *not* exposed in the alpha UI (segmenter choice hidden).

The CLI is not replaced. The web app talks to the same FastAPI surface the CLI drives; the CLI remains the power-user/ops tool.

2. Personas (from PMF research)

- **Coach / academy (primary, B2B, India):** batches of match recordings, wants rally review, per-player reels for students/parents, per-rally stats. Already records on static cameras.
- **Enthusiast (B2C funnel):** records own matches, wants a highlight reel + basic stats, shares with friends. Tripod discipline is the known adoption risk ? the UI must teach recording guidelines *before* upload, not fail videos after.

3. ALPHA ? "quick & dirty" (goal: 10?25 invited testers, end-to-end, zero hand-holding)

Stack: React + Vite + Tailwind SPA, deployed per HOSTING_PLAN (static hosting, $0). API = existing FastAPI on the GPU box via tunnel at `api.khelsutra.guru`. **Auth:** Cloudflare Access email-allowlist (OTP), free ?50 users, **zero auth code** ? backend reads the authenticated-email header for per-user scoping (alpha-grade tenancy: header is trustworthy because the tunnel only accepts Access-authenticated traffic).

Alpha screens (6)

#	Screen	What it does	Backed by
A1	**Matches** (home)	User's videos, status chip per video (verdict ?/??/?), upload CTA	`videos` + owner scoping (B4)
A2	**Upload & Process**	Drag-drop; inline recording-guidelines checklist (pre-upload education from VIDEO_RECORDING_GUIDELINES); chunked upload; then auto-starts processing	B2 upload endpoint ? B1 job
A3	**Job progress**	Stage list (validation ? segmentation ? AI confirm ? report) with live status; verdict banner on completion	B1 `GET /api/jobs/{id}` poll
A4	**Rally review** ?	Video player + rally list; click rally ? seek & play that span; per-rally **Accept / Reject / nudge start?end ±0.5 s**; bulk "all good"; chips: duration, shot count (+confidence)	B6 correction endpoints ? `candidate_segments.human_label/notes` ? **the corpus engine**
A5	**Highlights**	Select accepted rallies ? name reel ? compile ? download; list of past reels	`/api/compile` + B3 download endpoint
A6	**Report**	Renders the customer-audience report (verdict, notes, FAQ refs); "nerd view" toggle shows technical.md	existing reporting artifacts

Backend work the alpha actually requires (honest dependency list)

ID	Item	Notes
B1	**Async job model ? DEFERRED_HARDENING_PLAN #16**	The committed next PLATFORM slice; `POST /api/process` ? 202 + `job_id`, `jobs` table (user_version 1?2), ThreadPoolExecutor, `GET /api/jobs/{id}`. **Approving this proposal = greenlighting #16.** A 5?60 min blocking HTTP call cannot sit behind a tunnel UI.
B2	**Chunked upload endpoint**	`POST /api/upload` accepting ?90 MB parts (free-	

tier proxy body limit is 100 MB), assemble ? MD5 ? register under `allowed_video_root/<user>/`. Size cap (e.g. 4 GB) + mp4/mov allowlist. |
B3	**Download endpoint**	`GET /api/highlights/{video_id}/{name}` streaming the compiled reel (today `/api/compile` returns a server-local path the browser can't reach).
B4	**Per-user scoping**	`owner_email` column on `videos` (migration), read from the Access header, filter every query. Alpha-grade, single DB.
B5	**Retire/replace static UI**	The SPA supersedes `backend/api/static` (which has a known `rallies` vs `play_segments` field bug ? don't fix, replace).
B6	**Correction-loop endpoints**	`PATCH /api/segments/{id}` ? accept/reject/boundary-nudge persisted to `candidate_segments.human_label`/`notes` (+ provenance: who, when). This is v1-scope "correction loop" made real.
B7	**Exposure hardening**	CORS narrowed to the app origin; basic rate limit; upload quota per user; `allowed_video_root` set; Gemini key rotated (already in NEXT_STEPS housekeeping).

Alpha non-goals (explicit)

No payments, no orgs/teams, no athlete profiles, no share links, no owned-model serving, no multi-sport selector (badminton only), no notifications, no mobile-perfect layouts (usable on phone, optimized for laptop), no conversational QA (deferred in roadmap).

Alpha effort estimate

B1 ? 2?3 sessions · B2?B7 ? 2?3 sessions · SPA (6 screens) ? 3?4 sessions ? **~8?10 working sessions** to first tester invite. Sequencing: B1 ? B2/B3/B4 ? SPA skeleton + A2/A3 ? A4 ? A5/A6 ? B7 ? invite.

4. BETA ? polished (goal: paying pilot academies)

Maps deliberately onto PLATFORM_ARCHITECTURE P1?P3 ? the UI roadmap and the platform roadmap are the same roadmap.

1. **Drop 1 ? Accounts & academies (P1):** real auth (Firebase Auth ? GCP-native, free tier ? or Clerk), academy workspace, coach invites athletes (**18+ only**, per guardrail), roles (owner/coach/athlete). Coach?athlete stays an **optional overlay** on the generic schema.
2. **Drop 2 ? Storage & worker (P2/P3):** direct-to-cloud resumable uploads (signed URLs ? bypasses proxy body limits entirely), reels served from object storage; control plane moves to Cloud Run; **GPU box becomes the first pull-based `byo_worker`** (outbound-only, exactly P3's security model). Email notification when a reel is ready.
3. **Drop 3 ? Review v2:** keyboard shortcuts (J/K/L, [/] nudge), frame-step, timeline minimap with rally bars, batch accept, rally tags, athlete-facing **expiring share links**, per-player aggregates and match-over-match trends (v1 stats only ? **no gait/biometrics**, GDPR Art. 9).
4. **Drop 4 ? Monetization & consent:** Razorpay per-match credits (INR; PMF: B2C $60?180/yr, academy $300?1,500/yr equivalents, tuned per-match for India), free-tier match allowance; **training-data opt-in**: per-user toggle default OFF + per-match consent record + adult-only attestation at upload (guardrails: minors excluded from training pool; separate explicit opt-in).

Beta effort: ~4?6 weeks part-time across the four drops; each drop independently shippable.

5. Decisions needed from owner

1. **Greenlight** alpha scope above (= greenlights DEFERRED_HARDENING_PLAN #16)?
2. **RESOLVED 2026-06-10:** product name = **KhelSutra Guru**; domain = **khelsutra.guru** (GoDaddy receipt, 3-yr registration + domain protection, renews 2029-06).
3. Per-user **training opt-in default OFF** ? confirm (recommended; conservative per guardrails).
4. Alpha invite list: the C13 interview academies + camera-pilot contacts?
5. SPA stack confirm: React + Vite + Tailwind (proposed) vs anything you prefer.
6. **RESOLVED 2026-06-10:** separate **private repo `khelsutra`** (owner-ratified ? the product's
   "heart and brain"): SPA in `web/`, control plane joins at beta; backend PRs B1?B7 stay in the
   indexer repo. Keeps the indexer's licensing/test discipline untouched.

Once ratified, decisions 1?6 get recorded in `docs/archives/decisions/DECISIONS.md` per convention.

20. user (2026-06-20T09:00:01.608Z)

Hosting Plan ? khelsutra.guru

Status: PROPOSAL ? awaiting owner ratification.
Date: 2026-06-10 · **Constraints:** budget $10?25/mo · GCP credits available · DigitalOcean
acceptable · alpha backend = owner's GPU box behind a tunnel · 10?25 alpha testers · registrar = GoDaddy.
Companion: `docs/UI_PROPOSAL.md`.

1. Recommendation in one line

Alpha: free edge + your box (Cloudflare DNS/Pages/Access/Tunnel ? FastAPI on the GPU box) at
$0/mo, saving GCP credits; **Beta: GCP control plane + your box as pull-worker** (Cloud Run +
GCS + Firebase Auth, credits-covered), keeping the edge where it's free.

Rationale for the Cloudflare pieces despite the GCP preference: GCP has no free equivalent of the
Tunnel + Access combo (IAP requires a load balancer, ~$18/mo standing cost), and the alpha needs
exactly three things GCP charges for but Cloudflare gives free: a secure tunnel to a home box with
a custom domain, email-allowlist auth (free ?50 users ? fits 10?25 testers), and static hosting.
Credits are better spent at beta on compute/storage, which is where GCP is excellent.

2. Alpha topology ($0/mo)

```
tester browser
   ?  https
   ?
Cloudflare edge (free): DNS + TLS + Access (email-OTP allowlist)
   ??? app.khelsutra.guru  ? Cloudflare Pages (React SPA; auto-deploy from GitHub)
   ??? api.khelsutra.guru  ? Cloudflare Tunnel (outbound-only `cloudflared` service on the box)
                                      ?
                                      ?
                              FastAPI :8000 on the GPU box (Windows)
                              ? Gemini segmenter path (no GPU in request path)
                              ? async jobs (#16), SQLite WAL, output/ on disk
```

```

- **No inbound ports opened** on the home network; `cloudflared` dials out. Home IP never exposed.
- **Auth:** Cloudflare Access in front of BOTH hostnames; backend trusts the `Cf-Access-Authenticated-User-Email` header (verify the Access JWT for rigor) for per-user scoping.
- **Upload limit:** free-tier proxy caps request bodies at **100 MB** ? SPA uploads in ?90 MB chunks (UI_PROPOSAL B2). Match videos (1?4 GB) flow fine as chunks.
- **Box offline = service offline.** Acceptable for alpha; SPA shows a friendly "processing engine offline" banner (a `/api/health` ping). Beta removes this with the queue.
- **Costs:** Pages, Tunnel, Access, DNS = $0. Only the Gemini per-run cost (~$0.05/match) and domain renewal.

## 3. Beta topology (GCP credits; ~$15?20/mo post-credits)

```
browser ??? Cloudflare edge (free, unchanged) ??? app.khelsutra.guru (Pages)
                                              ??? api.khelsutra.guru ??? Cloud Run (FastAPI control plane, scale-to-zero)
                                                                                       ? jobs table (Cloud SQL Postgres)
                                                                                       ? signed resumable URLs
                                                                                       ?
                                                                                  GCS buckets (uploads / reels, lifecycle ? cold)
                                                                                       ? pull jobs, outbound-only
                                                                                  GPU box = first `byo_worker` (PLATFORM P3)
```

- Uploads go **browser ? GCS directly** (resumable signed URLs) ? no proxy body limit, no home bandwidth for ingest; the box pulls only what it processes (WASB/owned-model jobs), Cloud Run handles Gemini-path jobs entirely in-cloud.
- Auth graduates to **Firebase Auth** (free; Google + email sign-in; India-friendly phone OTP later); Access allowlist retires.
- Post-credits estimate: Cloud Run ~$0?5 (free tier: 2 M req + 180 K vCPU-s/mo, scale-to-zero) · GCS 100 GB ? $2.60 + egress $0.12/GB (reel downloads ? watch this line) · Cloud SQL smallest ? $10 ? **~$15?20/mo**, inside budget. Egress lever if it grows: serve reels from Cloudflare R2 (zero egress) while keeping everything else GCP.

## 4. Alternatives considered

| Option | Monthly (alpha ? beta) | Pros | Cons |
|---|---|---|---|
| **Recommended hybrid** (CF edge + box ? +GCP) | **$0 ? ~$0 w/ credits** (~$15?20 after) | Cheapest at every phase; credits used where they matter; maps to PLATFORM P2/P3 | Two vendors (CF + GCP) |
| **All-DigitalOcean** | ~$11 ? ~$17 ($6 droplet + Spaces $5: 250 GiB + **1 TiB egress included**) | One simple vendor; Spaces' included egress is the best video-download deal; per-second billing | No credits; no scale-to-zero; still needs a tunnel-equivalent to reach the box (run `frp`/WireGuard on the droplet) |
| **All-GCP from day 1** | ~$18+ ? credits | Single vendor, credits immediately | LB+IAP standing cost just to gate an alpha; no free tunnel to the home box; credits burn on plumbing |
```

All-Cloudflare (Pages+Workers+R2)	~$0 ? ~$5	Zero egress on R2; cheapest forever	FastAPI doesn't run on Workers (API stays on box/Cloud Run anyway); ignores GCP credits

5. Domain setup ? GoDaddy ? Cloudflare DNS (one-time, ~15 min, you)

1. Create a free Cloudflare account ? **Add a domain** ? `khelsutra.guru` ? Free plan. Cloudflare imports existing records and shows **two nameservers** (e.g. `xxx.ns.cloudflare.com`).
2. GoDaddy ? **My Products ? khelsutra.guru ? Manage DNS ? Nameservers ? Change ? "I'll use my own nameservers"** ? paste the two Cloudflare nameservers ? Save. Propagation: minutes to 24 h. (Registration/renewal stays at GoDaddy; only DNS moves. Optional later: transfer the registration to Cloudflare Registrar for at-cost .guru renewals.)
3. Records (I manage after takeover): `app` ? Pages project; `api` ? `<tunnel-id>.cfargotunnel.com` (created automatically by `cloudflared`); apex redirect ? `app`.

6. What I need to take over management

Access	Scope	Used for
Cloudflare API token (you create: dash ? My Profile ? API Tokens)	Zone:DNS:Edit, Account:Pages:Edit, Account:Access:Apps:Edit, Account:Tunnel:Edit ? scoped to this zone	DNS records, Pages deploys (wrangler), Access policies, tunnel config
GitHub repo `khelsutra` (private)	Pages connects once (dashboard, you; build root = `web/`) ? every push auto-deploys	I write the SPA in your Projects folder; commits pushed = deployed
On the box (one-time, I hand you exact commands)	`cloudflared` installed as a Windows service + `config.yml`	The tunnel itself
GCP (beta only)	A `khelsutra` project + service account (Cloud Run admin, Storage admin, Cloud SQL admin)	Control-plane deploys

Day-to-day flow after setup: I build ? commit ? push ? Pages deploys the SPA automatically; API/tunnel/DNS changes via the scoped token. Secrets (Gemini key, Access policy) never live in the repo; `.env` on the box + Cloudflare/GCP dashboards.

7. Go-live security checklist (alpha)

Access allowlist ON for both hostnames ? CORS narrowed to `https://app.khelsutra.guru` ? rate limit + per-user upload quota ? `security.allowed_video_root` set ? upload size cap + extension allowlist ? **rotate the Gemini API key** (NEXT_STEPS housekeeping ? it was shared in chat) ? confirm no router port-forwards ? weekly backup: SQLite DB + `output/` ? GCS bucket (rclone scheduled task; credits).

8. Decisions needed from owner

1. Ratify the **hybrid recommendation** (CF free edge now, GCP credits at beta)? Or force single-vendor (DO / GCP) despite the cost notes above?
2. OK to move **DNS to Cloudflare** (registration stays at GoDaddy)?
3. Beta reel storage: **GCS (credits)** vs **R2 (zero egress)** vs **DO Spaces (1 TiB included)** ? can defer until beta Drop 2.
4. Approve the access grants in §6 so I can run the setup end-to-end.

Ratified choices get recorded in `docs/archives/decisions/DECISIONS.md`.

21. assistant (2026-06-20T09:00:05.437Z)

Now let me look at the DECOUPLED_COMPUTE docs dir and the worker entry / storage base to ground the appliance + operator-console concept against what's actually shipped.

22. user (2026-06-20T09:00:07.933Z)

```
=== DECOUPLED_COMPUTE dir ===
docs/DECOUPLED_COMPUTE/01-EXISTING-PLANS-AND-GAP.md
docs/DECOUPLED_COMPUTE/02-COMPUTE-MAP-AND-CONTRACT.md
docs/DECOUPLED_COMPUTE/03-PLATFORM-OPTIONS.md
docs/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md
docs/DECOUPLED_COMPUTE/05-COST-ATTRIBUTION.md
docs/DECOUPLED_COMPUTE/06-RISKS-AND-OPEN-QUESTIONS.md
docs/DECOUPLED_COMPUTE/07-COMPUTE-ABSTRACTION.md
docs/DECOUPLED_COMPUTE/HANDOFF.md
docs/DECOUPLED_COMPUTE/README.md
=== HANDOFF? ===
DECOUPLED_COMPUTE
HARDENING_LOOP_HANDOFF.md
=== worker __main__ head ===
"""Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.

    python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
        [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
    python -m backend.worker train ...        # Gen-0 train-from-bucket (follow-up)

The URI scheme selects the storage backend via ``backend.storage`` (a bare path / ``local://``
is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing local runs
are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini key) ?
the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).

This is a thin orchestrator: it composes the SAME building blocks the ``backend.main`` CLI uses
(validator registry ? segmenter ? report), never a forked pipeline.
"""
from __future__ import annotations

import argparse
import csv
import json
import os
import sys
import tempfile
from typing import Any, List

from backend import storage
from backend.config import load_config
from backend.infrastructure.database import Database
from backend.pipeline.segmenters.base import segmenter_registry
from backend.pipeline.validators.base import registry as validator_registry
from backend.reporting import emit_indexer_report
from backend.results import Failure
from backend.utils.hashing import compute_video_id


def _coerce(v: str) -> Any:
    """Coerce a ``--set`` string value to bool/int/float where it round-trips, else str."""
    low = v.lower()
    if low in ("true", "false"):
        return low == "true"
    for cast in (int, float):
        try:
```

```
            return cast(v)
        except ValueError:
            pass
    return v


def _apply_overrides(config: Any, sets: List[str]) -> None:
    """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
    for kv in sets or []:
        key, sep, val = kv.partition("=")
        if not sep:
            raise ValueError(f"--set expects k=v, got {kv!r}")
        parts = key.split(".")
        obj = config
        for p in parts[:-1]:
            obj = getattr(obj, p)
        setattr(obj, parts[-1], _coerce(val))
```

23. assistant (2026-06-20T09:00:10.431Z)

Let me read the DECOUPLED_COMPUTE README and HANDOFF for the appliance/operator-console framing, plus the storage base for the reuse seam.

24. user (2026-06-20T09:00:11.930Z)

```
# Decoupled Compute ? Cloud-Native GPU for Train + Inference

**Status:** ? PROPOSAL / research ? for owner review. **No code yet.**
**Date:** 2026-06-20 · **Owner:** avidullu
**Lifecycle:** *Critical active project.* When delivered, **move this whole directory to `docs/archives/past_projects/`** (keeps the strategy tree clean).

> **What this directory is.** A single, unified place that answers one question: *can we lift the
> rally-segmentation pipeline off the owner's local Windows/WSL box and run it on a **rented,
> credit-budgeted, cloud-native GPU** ? driven by an API key ? that does **both** (a) **retrain** the
> rally model from a **bucket of videos** and (b) run **inference on new uploads** ? and **bill each
> video / each training run exactly**, including for a non-owner user?* Short answer: the *design*
> already exists across our strategy docs; the *build* (a container/VM image, a real bucket, a chosen
> provider, a cost ledger) does not. This directory inventories what's planned, maps the real compute,
> compares platforms, and lays out a cheapest-first build & test plan that fits the **$200** budget.

---

## North Star

A **decoupled** pipeline where the **control plane** (a thin API that accepts jobs and reports status)
is separated from the **data/compute plane** (the heavy bytes + GPU work), which runs in a **rented
cloud GPU** reading from and writing to an **object-store bucket**. Two verbs:

- **`train`** ? consume a bucket of videos + labels ? produce versioned model weights back to the bucket.
- **`infer`** ? a new uploaded video ? rally segments + highlight reel + an eval report back to the bucket.

```

?with **exact per-video (inference) and per-run (training) cost attribution** so the cost can be
charged to whoever ran the job.

TL;DR (read this, then dive into the numbered docs)

1. **Yes ? decoupling is already designed, in depth.** [`PLATFORM_ARCHITECTURE.md`](../PLATFORM_ARCHITECTURE.md)
   is a control-plane/data-plane split with a `ComputeBackend` registry (`local | hosted | byo_worker`)
   and a `StorageBackend` registry (S3/GCS/Azure via `fsspec`); [`HOSTING_PLAN.md`](../HOSTING_PLAN.md)
   already names *"GPU box = first `byo_worker`"* with GCS buckets + signed-URL uploads; §5A specifies
   training on a rented GPU via `ms-swift`. ? **[01](01-EXISTING-PLANS-AND-GAP.md)**.
2. **What's missing is everything that turns those seams into the North Star:** a container/VM image,
   a **chosen** provider + an **API-key/provisioning** flow, a **real bucket** (today it's OneDrive
   auto-sync ? a proto-bucket), a **download-to-local** shim, path de-hardcoding, a **bucket-triggered
   retrain**, a **remote inference endpoint**, and **cost metering**. ? **[01](01-EXISTING-PLANS-AND-GAP.md)**, **[02](02-COMPUTE-MAP-AND-CONTRACT.md)**.
3. **$200 is comfortable ? for the *right* scope.** It easily covers the **inference pipeline + Gen-0
   head training** (numpy-class, trains in minutes on a small GPU) decoupled onto one cheap GPU. It does
   NOT cover the **Gen-2 VLM fine-tune** (ms-swift/QLoRA, multi-GPU, *"the largest open risk,
   currently unquantified"*) ? that stays explicitly **out of scope** for this budget. ? **[03](03-PLATFORM-OPTIONS.md)**, **[06](06-RISKS-AND-OPEN-QUESTIONS.md)**.
4. **Docker is optional.** Serverless/Cloud Run paths need an image; a **DO Droplet** or **SkyPilot**
   path can run the repo in a plain `venv`/`conda` with no container. We recommend a container *where you
   go serverless* and a VM/SkyPilot path otherwise. ? **[03](03-PLATFORM-OPTIONS.md)**.
5. **Per-video billing falls out of a serverless per-invocation metering model + a cost ledger on the
   already-shipped `jobs` table.** ? **[05](05-COST-ATTRIBUTION.md)**.
6. **Cheapest-first: M0 costs $0** ? a CPU container/venv runs the **committed synthetic, video-free
   regression fixtures** ([`golden_fixtures.py`](../../backend/eval/golden_fixtures.py)) in CI, no GPU,
   no WSL. Only spend a dollar at M1. ? **[04](04-BUILD-AND-TEST-PLAN.md)**.
7. **The WSL detectors run seamlessly off the Windows box ? and we prove it on DigitalOcean.** The
   `DetectorRunner` ABC is already built for the swap (WSL plumbing is an isolated mixin; the `Frame,
   Visibility,X,Y` CSV is the parity contract); a DO GPU is Ubuntu+CUDA = your WSL conda env, **minus
   WSL**. A dedicated **M3.5 parity test on DO** is the guarantee, and it doubles as the
   `DESKTOP_APP_PLAN` P0 spike. ? **[02](02-COMPUTE-MAP-AND-CONTRACT.md)**, **[04](04-BUILD-AND-TEST-PLAN.md)**, **[06](06-RISKS-AND-OPEN-QUESTIONS.md)**.

The budget tension you must decide (see [03](03-PLATFORM-OPTIONS.md) + [06](06-RISKS-AND-OPEN-QUESTIONS.md))

You have **$200 in DigitalOcean**, and you're open to **GCS + a Cloud-Run-like** architecture. These pull
in different directions for the *side goal* (exact per-video billing):

- **DigitalOcean** has the credits + **Spaces** (S3-compatible, **1 TiB egress included** ? the best
  video-download deal), but **no true serverless GPU**, so per-video metering is coarser (you bill
  wall-clock on a started/stopped GPU Droplet).
- **GCP Cloud Run (GPU) / RunPod Serverless** give **scale-to-zero + exact per-invocation GPU-seconds**
  ? which *is* your per-video bill ? but the $200 isn't DO credit there (GCP's $300/90-day trial is the
  clean free tier; RunPod's "free credit" is small).

Recommended split: prototype M0?M2 on **DO (use the credits) + DO Spaces**; stand up the M3 inference
endpoint where per-invocation metering is native (Cloud Run GPU / RunPod Serverless). Final call is
owner's ? tracked in [06](06-RISKS-AND-OPEN-QUESTIONS.md).

Contents

Verification status (checked against code 2026-06-20)

These load-bearing facts were confirmed against the current tree (not just the older strategy docs):

- ? **Async-job control plane shipped** ? `jobs` table at [`database.py:188`](../../backend/infrastructure/database.py), worker at [`api/job_worker.py`](../../backend/api/job_worker.py), `/api/process` ? 202 + `job_id`.
- ? **`jobs` table has no cost/tenant columns yet** (cols: id, video_path, segmenter, player_pool, max_frames, status, policy, result, progress, timings) ? so the cost-ledger delta in [05](05-COST-ATTRIBUTION.md) is net-new.
- ? **Gen-0 harness exists** ? [`training/gen0/harness.py`](../../training/gen0/harness.py) (one-command train?eval?gate).
- ? **Committed synthetic, video-free regression fixtures exist** ? [`backend/eval/golden_fixtures.py`](../../backend/eval/golden_fixtures.py) ? M0 is a genuine $0 CI gate.
- ? **`"output"` path still hardcoded** ? [`trajectory_hybrid.py:196,272`](../../backend/pipeline/segmenters/trajectory_hybrid.py) (a real bucket-relocation blocker).
- ? **No Dockerfile anywhere** in any of the three repos ? the container is greenfield.
- ? **Collector keys by human `video_id`, not MD5** ? `curate.py`; the export manifest omits an `md5` field, which the MD5-keyed consumer needs (see [02](02-COMPUTE-MAP-AND-

CONTRACT.md)).

Provenance

Built from a 13-agent research workflow (?970k tokens, 5 plan-harvest + 3 compute-map + 2 cloud-provider agents, then synthesis + an adversarial completeness critic), cross-checked against the owner's strategy docs and the live code. Cost/price figures are **approximate ? verify current provider terms before committing budget.**

25. user (2026-06-20T09:00:15.562Z)

DECOUPLED_COMPUTE ? Session Handoff / Resume-Here

Updated: 2026-06-20 · **Read this first to resume.** Companion: this whole directory + `docs/SETUP_NEW_MACHINE.md`.

TL;DR ? where we are

The **entire bucket-driven `train ? infer` pipeline runs end-to-end on a CPU box**, with the GPU **mocked** (deterministic stub). Validated on a DigitalOcean droplet reading/writing **DO Spaces**, driven remotely over SSH. Everything is merged to `master` (PRs #246?#254); the suite is **936 green**.

The ONE remaining piece for *real* (non-mock) training is the **Linux-native WASB `DetectorRunner` (M3.5)** running on a **GPU box** ? it produces *real* shuttle trajectories instead of the synthetic stub. Once it exists, the **same `worker train` / `worker infer` commands run unchanged**; only the trajectory source flips.

What's built (all merged to master)

PR	What	Key files
#249	Storage layer ? `local` + **S3 (AWS/DO Spaces/R2/MinIO)** + GCS, URI-driven; gdrive stub	`backend/storage/`
#251, #254	**Worker** `python -m backend.worker {infer|train}` ? pull from storage ? pipeline ? push	`backend/worker/__main__.py`
#247	CPU-mock detector seam (`RALLY_DETECTOR_IMPL=stub` / `indexing.detector_impl=stub`)	`backend/pipeline/detectors/stub_runner.py`
#253	GPU provisioning ? NVMe scratch + **native WASB setup** + port runbook	`deploy/digitalocean/{bootstrap,mount_scratch,gpu_setup}.sh`
#252	Cross-platform canonicalization (Linux/macOS, not just Windows)	(segmenters, compiler, doctor.py)
#246	The plan directory	`docs/DECOUPLED_COMPUTE/`

Verified e2e (CPU, no GPU, no Gemini key): `worker train` pulled 7 labels from Spaces ? `synth_trajectory_from_labels` (label-consistent CPU mock) ? `training.gen0.harness` (shelled out ? `backend` must not import `training`) ? LOVO over 7 videos (F1 1.000 on the mock, gate=no-ship by design) ? `weights/` to Spaces. `worker infer` pulled a real video ? stub ? `outputs/<md5>/` to Spaces.

Resources & access (current)

- **CPU droplet:** `168.144.126.176` (region `blr1`, CPU-only, no NVMe). SSH: `ssh -i ~/.ssh/do_rally root@?`
  (key generated locally; pubkey added to DO). Code + venv at `~/rally`; Gemini key at `/root/.keys/GEMINI_API_KEY`.

- **DO Spaces:** bucket **`rally-e2e`**, region **`sgp1`**, endpoint `https://sgp1.digitaloceanspaces.com`.
  ? **The Spaces key was exposed in chat ? ROTATE it.** Creds live on the box at `~/.aws/credentials` (never in repo).
- **Corpus staged in Spaces:** `videos/` (8 .mp4) + `labels/` (7 `.rallies.csv`) + `weights/0.1.0-smoke/` + `outputs/`.
- **Drive corpus** (source): folder `1sHGL7iacnmM80ChT-p2cKy638gK504T_` (gdown-able by file-id). 7 `[Done]` = video+labels;
  Video 1 `GX010135_proxy.mp4` is the held-out (non-Done) test.
- **Helper scripts on the box:** `/root/do_e2e.sh` (reproduces the train+infer e2e), `/root/do_spaces_config.json`
  (`storage.s3` sgp1 + `skip_ai_handoff` + `detector_impl=stub` + permissive validation), `/root/do_stage_{labels,videos}.sh`.

Reproduce the working e2e (today, CPU)

```bash
ssh -i ~/.ssh/do_rally root@168.144.126.176 'bash ~/do_e2e.sh'
# train (7-video LOVO) + infer (real video) from Spaces ? artifacts back to Spaces.
```

NEXT BUILD ? real trajectories (M3.5)

1. **GPU box** (owner): spin up a DO GPU Droplet **in `sgp1`** (co-located with the bucket) ? `SETUP_NEW_MACHINE.md` §2
   has the `doctl` command (verify slugs). Provide a DO API token if you want it `doctl`-provisioned. Add the same SSH key.
2. **Port** (`SETUP_NEW_MACHINE.md` §9): git-archive ? scp ? `mount_scratch.sh` ? `bootstrap.sh --gpu` ? `gpu_setup.sh`
   (native WASB conda env) ? secrets (rotated Gemini key + Spaces creds).
3. **Build `NativeWasbRunner(DetectorRunner)`** in `backend/pipeline/detectors/` ? sibling to `WasbRunner` but **without**
   `WslCommandMixin`: run WASB in-process/native (no `wsl`/`conda activate`/`to_wsl_mnt_path`), device `cuda/mps/cpu`,
   weights via env (not `~/models`), emit the identical `Frame,Visibility,X,Y` CSV. Select it via the existing
   `detector_impl` seam (`trajectory_hybrid._resolve_runner`). Scout details: `06-RISKS-AND-OPEN-QUESTIONS.md` (E1?E6 in the
   merged workflow plan) + `docs/TRACKNET_WSL_SETUP.md`.
4. **Acceptance gate:** byte-parity ? same clip ? WSL CSV (Windows) == native CSV (Linux) + identical candidate windows.
5. **Wire real extraction into `worker train`/`infer`:** today `worker train` uses `synth_trajectory_from_labels`; add a
   real-extract path that runs the detector per video ? trajectory CSV ? manifest. The harness + push are unchanged.

Gotchas / open items

- **`backend` ? `training`** one-way wall ? the worker **shells out** to `python -m training.gen0.harness`.
- **Gen-0 ship-gate F1@tIoU is provisional (#172)** ? advisory; do **not** hard-gate on it.
- **WASB-C3** weight provenance is an open **legal** item before sharing any trajectories/model externally.
- WASB (torch 1.11) and TrackNet (TF) must stay in **separate conda envs** (segfault if co-installed).
- The box still has MinIO (`:9000/:9001`) + a GCS emulator (`:9023`) running from earlier provider tests (harmless).
- Two-provider goal (GCS in a Bangalore-ish region, parallel hosting) and the `/srv/khelsutra-site` web-hosting verify
  + a real-Gemini-detection smoke are **queued, not done**.

26. assistant (2026-06-20T09:00:21.168Z)

I now have enough grounding. Let me quickly verify the storage base seam (the StorageBackend ABC) and the parity CI patterns to cite reuse precisely.

27. user (2026-06-20T09:00:22.541Z)

```
=== storage/base.py head ===
"""The ``StorageBackend`` ABC ? one interface for local + object-store + Drive.

Mirrors the codebase's ABC+Registry idiom (cf. ``SegmenterRegistry``). Concrete backends
live in sibling modules and lazy-import their cloud SDK so ``import backend.storage`` never
pulls ``boto3`` / ``google-cloud-storage`` / ``pydrive2``. See ``PLATFORM_ARCHITECTURE.md``
§3.1 and ``docs/DECOUPLED_COMPUTE/``.
"""
from __future__ import annotations

import os
from abc import ABC, abstractmethod
from typing import Iterator

from backend.storage.ref import StorageRef, StorageStat


class StorageBackend(ABC):
    """Backend-agnostic object storage. ``fetch_to_local`` + ``put`` are the two the
    worker dispatcher needs; the rest support listing/checkpointing/serving."""

    @abstractmethod
    def fetch_to_local(self, ref: StorageRef, dst: "str | os.PathLike[str] | None" = None,
                       *, overwrite: bool = False) -> str:
        """Make the object available as a local file; return its local path. For the
        ``local`` backend this is an identity passthrough (the object is already local)."""

    @abstractmethod
    def put(self, local_path: "str | os.PathLike[str]", ref: StorageRef, *,
            content_type: "str | None" = None) -> StorageRef:
        """Upload/copy a local file to ``ref``; return ``ref``."""

    @abstractmethod
    def exists(self, ref: StorageRef) -> bool: ...

    @abstractmethod
    def stat(self, ref: StorageRef) -> StorageStat: ...

    @abstractmethod
    def list(self, prefix: StorageRef) -> Iterator[StorageRef]:
        """Yield a ``StorageRef`` per object under ``prefix``."""

    @abstractmethod
    def delete(self, ref: StorageRef) -> None: ...

    def signed_url(self, ref: StorageRef, *, ttl: int = 3600, method: str = "GET") -> str:
        """A time-limited URL handing the bytes to a browser/LLM. Not every backend can
        sign (``local`` cannot; Drive only emulates) ? callers must treat
        ``NotImplementedError`` as expected and fall back to ``fetch_to_local``."""
        raise NotImplementedError(f"{type(self).__name__} cannot produce signed URLs")
=== storage/__init__.py (factory) ===
"""Pluggable storage layer: local (default) + S3/GCS first-class buckets + Drive (follow-up).

URI-driven (see ``ref.parse_uri``). The package import is cheap ? cloud SDKs are imported
lazily, only when a non-local URI is actually used (install the ``storage-s3`` / ``storage-gcs``
/ ``storage-gdrive`` extras). The default ``local`` backend needs no third-party deps and
preserves today's behaviour byte-for-byte.

Worker/dispatcher entry points speak URIs via the thin helpers ``fetch`` / ``put`` /
``list_uris`` / ``get_storage``; ``local://`` (and bare paths) make ``fetch`` an identity
```

```
passthrough. See ``docs/DECOUPLED_COMPUTE/``.
"""
from __future__ import annotations

import os
from typing import Any, Dict, Iterator, Optional

from backend.storage.base import StorageBackend
from backend.storage.ref import StorageRef, StorageStat, parse_uri

__all__ = [
    "StorageBackend", "StorageRef", "StorageStat", "parse_uri",
    "get_storage", "fetch", "put", "list_uris",
]


def _backend_class(name: str):
    """Lazy-resolve a backend class so cloud SDKs aren't imported until needed."""
    if name == "local":
        from backend.storage.local import LocalStorage
        return LocalStorage
    if name == "s3":
        from backend.storage.s3 import S3Storage
        return S3Storage
    if name == "gcs":
        from backend.storage.gcs import GCSStorage
        return GCSStorage
    if name == "gdrive":
        from backend.storage.gdrive import GDriveStorage
        return GDriveStorage
    raise ValueError(f"unknown storage backend: {name!r} (expected local|s3|gcs|gdrive)")
```

28. assistant (2026-06-20T09:00:24.906Z)

I have all the grounding I need. Let me look at the parity CI workflow briefly to cite the reuse pattern precisely.

29. user (2026-06-20T09:00:25.505Z)

```
# Live-host parity ? the site runs on TWO hosts (Cloudflare Worker + the DO droplet's
# Node/SQLite twin) that must serve byte-identical pages. The unit suite proves the code
# works on each host; it CANNOT see that the two LIVE URLs have drifted (e.g. a merge
# auto-deployed Cloudflare but the droplet wasn't re-synced from git). This runs the
# existing scripts/parity-check.sh against both live URLs on a schedule ? NOT on PRs, so
# it never blocks development; a failure surfaces real production drift to the owner.
name: live-parity

on:
  schedule:
    - cron: "17 6 * * *" # daily ~06:17 UTC
  workflow_dispatch: {} # and on demand

# Avoid overlapping runs if a manual dispatch races the cron.
concurrency:
  group: live-parity
  cancel-in-progress: false

jobs:
  parity:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Compare the two live hosts byte-for-byte
        env:
          CF_URL: https://khelsutra.guru
```

```
              VM_URL: http://168.144.126.176:8787
            run: bash site/scripts/parity-check.sh

```

30. assistant (2026-06-20T09:00:28.504Z)

I have comprehensive grounding now. Let me compile the structured output.

31. user (2026-06-20T09:01:43.258Z)

Structured output provided successfully

32. assistant (2026-06-20T09:01:46.963Z)

Mapping complete. Structured output returned covering the four requested areas: the khelsutra UI surface (site/ public marketing vs web/ alpha SPA scaffold vs control-plane/ beta), the gated-app-vs-public-page boundary for the operator console, how per-rally selection is one screen of the broader console, the exact PROJECT_DOC_TEMPLATE structure to follow, concrete reuse opportunities, and path:line file refs.